

DSAIL- HW8

Tomas Salaz

March 23, 2026

Problem 1

Part A

Let X_n be an Indicator RV where $X_n = 1 \rightarrow$ Balloon n flew over Ferris.

$E[X_n] = Pr(X_n = 1) = 3/n$, from the Problem statement.

By LOE, $E[\# \text{ Balloons over Ferris}] = 3/n \cdot Kn = 3k = 3$ since $k = 1$.

Part B

$Pr[\geq 9 \text{ over Ferris on Day 1}]$ is the probability we are looking for. Let $X_k =$ Total number of Balloons over Ferris on day k . We know $E[X_k] = 3$.

By Markovs, $Pr[\geq 9] \leq E[X_k]/9 = (3)/9 = 1/3$

Part C

Union Bound says $Pr(A_1 \cup A_2 \cup \dots \cup A_n) \leq \sum_{i=1}^n Pr[A_i]$

$Pr[\text{One Severe Day}] = \sum_{i=1}^k Pr[A_k] = \sum_{i=1}^k 1/k^3 = k \cdot 1/k^3 = 1/k^2$

Part D

We Are looking for $Pr[\text{Empty Launch Sites}]$. Let X_n be Indicator RV st $X_n = 1 \rightarrow$ Site n is empty.

$Pr[X_n = 1] = (1 - (1/n))^n$

We then take the sum over the n sites so

$\sum_{i=1}^n Pr[X_n = 1] = \sum_{i=1}^n (1 - (1/n))^n = n \cdot (1 - 1/n)^n$

Problem 2

Part A

Proof by Counter Example

Taking the Naive Algorithm, and considering the array $nums = [1, 3, 1, 2]$.

The Naive Algo, Firstly Chooses idx 2 generating 6 coins, then there is a tie since the array is now $[1, 3, 2]$ so assuming the algo takes the 1st index when there is a tie, we generate 6 coins leaving the array $[1, 2]$ once again there is a tie so we take the 1st index leaving 2 and the last move also generates 2, for a total of 16 coins. Even when taking the 2nd index when there is a tie generate 16 coins.

A better order would be to take the idx=0 for 3 coins, leaving $[3, 1, 2]$, then we take the 1 for 6 coins leaving $[3, 2]$, then we can take the 2 for another 6 coins, leaving 3 coins for a total of 18 coins. Thus the naive is not optimal.

Part B

Let $C(i, j)$ be the Maximum coins generated by popping balloons between the indexes i and j , not including i and j themselves. We can 'pad' the array with 1's so $num[0]$ and $num[n+1] = 1$, so the subproblem is defined as $C(0, n+1)$

Part C

To compute $C(i, j)$ we try every balloon k , between the bounds as the last balloon popped in the range. If K is the last balloon popped then :

Before Popping k , we already popped everything between i and k

We also already popped everything between k and j

When we pop k , Since everything else is gone, we get the result $nums[i] * nums[k] * nums[j]$

Putting it all together, we get that

$$C(i, j) = \max \forall k ; i < k < j \text{ of } : C(i, k) + nums[i] * nums[k] * nums[j] + C(k, j)$$

This implies that we take the k or index in the range that generates the Max number of Coins

Part D

The Memoization for this DP would have to be filled bottom up, by gap size $(j - i)$. We start with the Gap=2 since that is the smallest subproblem with a balloon between i and j .

The runtime of the DP is $O(n^3)$ since there table is $n \times n$ there are n^2 subproblems each with another n amount of work to do.

Other Parts We pad the Array to get $nums = [1, 6, 1, 2, 7, 1]$

$C(i, j)$ = Max coins between i and j . With the Gap=2, we can assume k to be in the range $[1..4]$, with these values we can fill out the table for the following tuples:

$(0, 2), (1, 3), (2, 4), (3, 5)$

Then we increase the gap size to 3 and can fill out the following tuples:

$(0, 3), (1, 4), (2, 5)$

With Gap size 4, These Tuples: $(0, 4), (1, 5)$

With Gap size 5 we only have 1 tuple: $(0, 5)$

	j=0	1	2	3	4	5
i=0	—	0	6	24	138	145
1		—	0	12	96	138
2			—	0	14	21
3				—	0	14
4					—	0
5						—

For the 2nd table, we use it similarly to how we used it for the Matrix Chain. We have to read from $(0, 5)$ then trace back until we get to the diagonal with the smallest gap.

Problem 3

The notation is $Job = (StartTime, EndTime)$ None are optimal and are all proved by Counter Example.

Part A

Let the set of Jobs be $A = (0, 5)$, $B = (4, 5)$ and $C = (5, 10)$.

If these are the jobs in the queue, the Shortest Job is B , but it conflicts with both A and C , so this is not optimal.

The Optimal Solution is Jobs A and C

Part B

Let the set of Jobs be $A = (0, 50)$, $B = (5, 10)$ and $C = (15, 45)$.

If this is the case only Job A get ran and the other are discarded so this is not optimal.

The Optimal Solution is B and C

Part C

Let the set of Jobs be $A = (0, 50)$, $B = (5, 10)$ and $C = (15, 45)$.

If this is the case, again we only run Job A and the other get discarded so this is not optimal.

The Optimal Solution is B and C

Part D

Let the set of Jobs be $A = (0, 1)$, $B = (0, 2)$, $C = (0, 2)$, $D = (1, 3)$, $E = (2, 4)$, $F = (3, 5)$, $G = (4, 6)$, $H = (4, 6)$, $I = (5, 6)$

Given the set above we can count the conflicts:

A has 2, BC

B has 3, ACD

C has 3, ABD

D has 3, BCE

E has 2, DF

F has 3, EGH

G has 3, FHI

H has 3, FGI

I has 2, GH

With the Greedy algo we execute A, E, I in since they have the least amount of conflicts.

An Optimal solution given the set could run $A(0, 1) \rightarrow D(1, 3) \rightarrow F(3, 5) \rightarrow I(5, 6)$, we can run 4 jobs as opposed to 3.

Problem 4

Part A

To prove that the greedy solution from class doesn't do good with the weighted version, we can let there be 2 jobs:

$A = (0, 3)$ with weight 1

$B = (2, 5)$ with weight 100

Since the Greedy Algo from Lecture chooses the Job that has the earliest Finish Time. In this case we choose Job A and discard Job B, but we only have a weight of 1.

The optimal solution would obviously be to choose Job B with weight 100.

Part B

To create the optimal subproblem for this algorithm, we need to sort the Jobs by Finish Time. We can assume that Job J finishes no later than Job $J + 1$.

Let $m(j) = \text{Max Total Weight achievable in the set of Jobs } [a_1, a_2 \dots a_n]$.

For any Job a_j , there is a binary choice when making the optimal solution:

a_j IS NOT included in the solution. So the best we can do is what was optimal in the previous steps before, from $[1..j - 1]$. Which is the same as $m(j - 1)$

a_j IS included in the solution. Since this Job is chosen, we gain $w(a_j)$, the weight of the job. To avoid the conflicts of this job, we look go to the Finish Time of Job a_j since they are sorted by Finish Time, So we can define x_j to be the index of the latest job that finishes before a_j starts. This is the same as $m(x_j)$.

For Each Job j , $x_j =$ the largest index $i < j$ such that jobs i finishes before job j starts. This is the same as $\text{finish}(a_i) \leq \text{start}(a_j)$. $x_j = 0$ if this doesn't exist. To compute, For each j , iterate backwards through the Jobs from $[1..j - 1]$ and find the latest one that has no conflicts. The iterations(loop) take $O(n)$ time and each iteration take another $O(n)$ time for a total runtime of $O(n^2)$.

The Recurrence would be:

$$m(0) = 0$$

$$m(j) = \max(m(j - 1), w(a_j) + m(x_j))$$